CS 214: Data Structures

Abstraction and Implementation-Related Issues
(interesting)

Slide contents follow

Kruse and Leung "Data Structures & Program Design in C"

Prepared by:

Waleed A. Yousef, Ph.D.

© Waleed A. Yousef 2008

** How to use a queue, a stack, and other data structures with the same element type in the same program?

Since the element type of the stack, queue, and the main program is the same, we need to be more structured define this common element type in a separate file `Global.h`. In this file we should have all the definitions that are common to all of the three modules;

```
                    /******Global.h******/
typedef struct elementtype{
        int    year;
        float  age;
        int    tmp;
}ElementType;

#define MAXELEMENTS 100

typedef ElementType QueueEntry;
#define MAXQUEUE MAXELEMENTS

typedef ElementType StackEntry;
#define MAXSTACK MAXELEMENTS
```

Elabsera

© Waleed A. Yousef 2008

`Stack.cpp` must include `Stack.h` and the latter includes `Global.h` why?

`Queue.cpp` must include `Queue.h` and the latter includes `Global.h`.

Finally, `Main.cpp` includes all the three header files; why?

However, this will cause a "redefinition" error since the definitions in `Global.h` will appear again in the other two included header files because they also include `Global.h`.

To resolve this problem, we need to start `Global.h` by:

```
#ifndef GLOBAL H
#define GLOBAL H
//here comes the previous definition of Global.h
#endif
```

These statements are "Preprocessor Commands" that are processed before compilation. More explanation will be provided in labs; you can also refer to your C language text book.

© Waleed A. Yousef 2008

Notice that:

- Each of the previous definition starts with "*… of elements of type T is a finite sequence of elements of T together with the following operations:*". it is apparent that Stacks, Queues, and Lists differ only in the set of operations defined on each of them.

- The method of implementation is NOT mentioned in the definition.

- The interface of the ADT (the header file in case of C programming, along with the pre- and post-conditions of the functions) defines the contract between users and implementers that provides a precise means of communicating what each can expect from the other.

© Waleed A. Yousef 2008

How can we selected among different implementations?

Should information about the efficiency of implementations be included in the interface so that the user chooses between different complexities (of course without letting him involved in the implementation details)?

Possible Solution:

Tell the user to write in `Queue.h` the following if he cares about memory utilization than speed

```
#define LIMITED_MEMORY_
```

Then, in `Queue.cpp` write the two implementations of Queue with the following pre-processor:

```
#ifdef LIMITED_MEMORY_
      //Write here the implementation of Linked Queues;
#else
      //Write here the implementation of Contiguous Queues;
#endif
```

© Waleed A. Yousef 2008

What about hiding the type definition from the header files?

Possible Solution: we define the stack to be pointer to struct; this struct will be defined only in the stack.cpp

```
    /*LinkedStackMoreHiding.cpp*/
#include <stdio.h>
#include <stdlib.h>

#include "Global.h"
#include "Stack.h"

typedef struct stacknode{
   StackEntry entry;
   struct stacknode *next;
}StackNode;//the same as before

struct stack{
   StackNode *top;
};//We do not use typedef here

void Push(StackEntry item, PStack ps){
//exactly the same code; we only use
the type PStack instead of Stack *
}
```

```
        /* Old Stack.h*/
typedef struct stack{
   StackNode *top;
}Stack;
```

```
        /* New Stack.h*/
typedef struct stack *Pstack;
```

```
        /*Main.cpp*/
#include "Global.h"
#include "Stack.h"
void main(){
PStack ps;//This statement
will compile correctly
since PStack is a pointer;
it does not matter a
pointer to what
```

© Waleed A. Yousef 2008

The only difference will be in the functions that change ps; namely, CreateStack and ClearStack.

```
/*Stack.h*/
typedef struct stack *PStack;

void CreateStack (PStack *);
```

```
/*Main.cpp*/
#include "Global.h"
#include "Stack.h"
void main(){
PStack ps;
CreateStack(&ps);
/*When ps is defined no structures is created yet; Therefore, CreateStack has to use malloc to create the top and other fields.*/
```

```
void CreateStack (PStack *pps){
  *pps=(struct stack *)malloc(sizeof(struct stack));
  (*pps)->top=NULL;
}
```

© Waleed A. Yousef 2008

What about using different element types in the same ADT? For example, `StackEntry1` and `StackEntry2`?

Imagine!! So far we cannot use:

- more than one stack with different homogeneous element type.
- or even, a single stack of non- homogeneous element types.

Solution 1: (but very un-smart)

- Combine the fields of the different possible types in one data type.
- Add an extra field, name it `etype`, and give it a code for every type.
- Then, it is the responsibility of the user level to check for `etype`.

```
typedef struct Elementtype{
    short int etype;
    Type0     data;
    Type1     voice;
    Type2
fax; }ElementType;

//Waste of memory for that
all the fields are reserved
and only one will be used
```

```
#define 0 DATA
#define 1 VOICE
#define 2 FAX
void main(){
ElementType e;

Pop(&e, &s);
switch (e.etype)
  case DATA:
  case VOICE:
...
```

© Waleed A. Yousef 2008

Solution 2: (unions to save memory)

```
typedef struct elementtype{
    short int  etype;
    union{
      Type0    data;
      Type1    voice;
      Type2    fax;
    }info;
}ElementType;

//The maximum of the fields will
  be reserved in the memory
```

```
#define 0 DATA
#define 1 VOICE
#define 2 FAX

void main(){
    ElementType e;
    Pop(&e, &s);
    switch (e.etype)
      case DATA:
         e.info.data ...
      case VOICE:
         e.info.voice ...
      case FAX:
         e.info.fax ...
...
```

© Waleed A. Yousef 2008

Solution 3 (much smarter): not only solves the issue but also allows commercializing `Stack.obj` without the dependency on `Stack.h`

Notice: `Push` calls `malloc(sizeof(StackNode))` and `StackNode` contains `StackEntry`, which was compiled **already** into `Stack.obj`. If the user defines another `StackEntry` in `Global.h` this will cause a runtime problem! (Try it home).

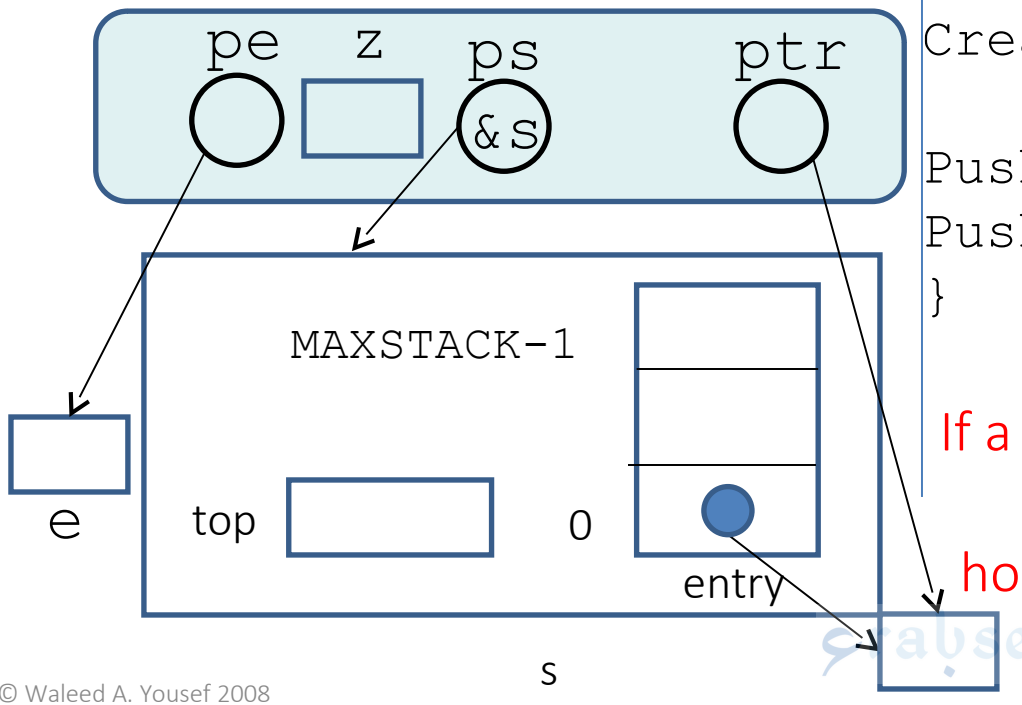
The solution, allows user for using both:

- more than one stack with different homogeneous element type.
- or even, a single stack of non- homogeneous element types.

© Waleed A. Yousef 2008

```
typedef struct stack{
    int top;
    void *entry[MAXSTACK];
}Stack;

void Push(void *pe, Stack *ps, int z){
    void *ptr=malloc(z);
    memcpy(ptr, pe, z);
    ps->entry[ps->top]=ptr;
    ps->top++;
}
```




```
void main(){          //User Level
Stack s;
char  e1;
int   e2;
...
CreateStack(&s);

Push((void*)&e1,&s,sizeof(e1));
Push((void*)&e2,&s,sizeof(e2));
}
```

What about Pop?

If a single type exists in the stack, Push is fine this way. However, if non-homogeneous elements exist we have to add a field of Element size.

© Waleed A. Yousef 2008

Solution 4 (most efficient solution): using C++ (OOP language) supporting the definition of an object of undetermined type known by Templates:

Implementation

```
template   <class   T>

class Stack{
private:
      int top;
      T *nodes;
public:
      Stack();
      void Push(T&);
      ...

      ~Stack();
};
```

User

```
...

Stack<char> s1;
Stack<int> s2;
```

Unfortunately, this requires:

- Knowledge of OOP
- Knowledge of C++,
- Which means delaying the Data Structures course two semesters.

© Waleed A. Yousef 2008

More subtle issues and some answers: We just open the door for deeper thinking and robust design

Much more of these issues are out of the scope of our course. They are classified as low-level Software Engineering aspects.

Also, solutions to these raised issues are language dependent. E.g., the definition of the structure completely disappeared from the header file by defining a pointer to an undefined structure. In some languages, e.g., Java, this is not available since there is no *pointers.*

However, the student should be aware of them at the level that we discussed here.

© Waleed A. Yousef 2008